

PROPOSAL TEKNIS

Sistem Manajemen Lab & Skrining Awal TBC

Mata Kuliah	Keamanan Informasi (KOM1315)
Kelompok	Kelompok 7 — Kelas K1
Anggota	Walid Nadirul Ahnaf (G6401231109)
	Yasir (G6401231091)
	Cokorda Satria (G6401231064)

1. DESKRIPSI SISTEM

1.1 Latar Belakang

Sistem Manajemen Lab & Skrining Awal Tuberkulosis (TBC) merupakan aplikasi backend berbasis REST API yang dikembangkan sebagai capstone project mata kuliah Keamanan Informasi. Sistem ini dirancang untuk mengelola alur kerja laboratorium patologi dalam proses skrining awal TBC melalui analisis citra histopatologi dengan bantuan kecerdasan buatan (AI).

Tuberkulosis merupakan penyakit menular yang disebabkan oleh *Mycobacterium tuberculosis*. Diagnosis TBC melalui pemeriksaan histopatologi jaringan memerlukan keahlian khusus dokter patologi untuk mengidentifikasi struktur khas seperti granuloma, sel datia Langhans (Langhans giant cell), sel epiteloid, dan area nekrosis. Sistem ini mengotomatisasi proses analisis awal menggunakan model AI, kemudian memfasilitasi validasi oleh dokter patologi sebelum diagnosis final ditetapkan.

1.2 Tujuan Sistem

Sistem ini memiliki tiga tujuan utama yang saling berkaitan:

- **Efisiensi Diagnostik:** Mengotomatisasi skrining awal citra histopatologi TBC menggunakan model AI untuk mendeteksi temuan patologis (granuloma, nekrosis, sel datia Langhans, sel epiteloid).
- **Keamanan Data Klinis:** Melindungi data sensitif pasien (NIK, nomor BPJS, citra medis, hasil diagnosis) dengan menerapkan prinsip security by design pada setiap lapisan sistem.
- **Kolaborasi Terstruktur:** Memfasilitasi workflow terstandar antara Operator Lab, Dokter Patologi, dan Administrator AI dalam proses validasi dan konsensus diagnosis.

1.3 Aktor Sistem & Peran

Peran (Role)	Kode Enum	Tanggung Jawab
Operator Lab	OPERATOR_LAB	Mendaftarkan pasien, membuat kasus pemeriksaan, mengunggah citra histopatologi, dan mengajukan kasus ke antrian validasi.
Dokter Patologi	DOKTER_PATOLOGI	Mereview citra dari antrian, memvalidasi hasil analisis AI per citra, membuat konsensus diagnosis akhir, dan menandatangani laporan klinis secara digital.

Admin AI	ADMIN_AI	Monitoring operasional modul AI, memantau status pemrosesan, dan mengelola konfigurasi sistem AI.
-----------------	-----------------	---

1.4 Alur Kerja Sistem

Sistem beroperasi mengikuti alur kerja linear yang terdefinisi melalui status kasus (CaseStatus):

- **PENDING_UPLOAD** → Operator Lab mendaftarkan pasien dan membuat kasus baru. Upload citra histopatologi melalui presigned URL ke Supabase Storage. Citra menjalani Quality Control (QC) otomatis.
- **AI_PROCESSING** → Setelah semua citra lulus QC, sistem AI memproses citra untuk mendeteksi dan mengkuantifikasi temuan patologis.
- **PENDING_VALIDATION** → Kasus masuk antrian review dokter patologi. Dokter memvalidasi hasil AI per citra dengan label kategoris klinis.
- **RESOLVED** → Dokter membuat konsensus final. Laporan klinis digenerate dan ditandatangani secara digital.

1.5 Tech Stack Backend

Layer	Teknologi	Versi / Keterangan
Runtime	Node.js	LTS ES Modules (type: module)
Language	TypeScript	^6.0.3, strict mode aktif (no any)
Framework	Express.js	^5.2.1
ORM	Prisma	^6.19.2 dengan PostgreSQL
Database	PostgreSQL	via Supabase (cloud-hosted, connection pooling PgBouncer)
Auth	JWT + bcrypt	jsonwebtoken ^9.0.3, bcrypt ^6.0.0
Validasi	Zod	^4.3.6 — schema-based validation
Storage	Supabase Storage	@supabase/supabase-js ^2.105.4 — presigned URL
Testing	Vitest	^4.1.5 dengan coverage V8

2. IDENTIFIKASI ASET

Berdasarkan analisis skema database (prisma/schema.prisma) dan source code sistem, aset-aset berikut diidentifikasi dan diklasifikasikan berdasarkan tingkat sensitivitasnya:

2.1 Aset Data (Data Assets)

2.1.1 Data Identitas Pasien - Model: Patient

Field	Tipe	Sensitivitas	Keterangan
no_induk (NIK)	String UNIQUE	SANGAT TINGGI	Nomor Induk Kependudukan
bpjs_number	String UNIQUE	SANGAT TINGGI	Nomor BPJS Kesehatan
name	String	TINGGI	Nama lengkap pasien
sex, age	Enum, Int	TINGGI	Jenis kelamin dan usia

2.1.2 Data Citra Medis - Model: Image

Field	Tipe	Sensitivitas	Keterangan
file_path	String	SANGAT TINGGI	Path file histopatologi di Supabase Storage akses via signed URL saja
magnification	Enum (X10/X40/X100)	SEDANG	Perbesaran mikroskop (X10, X40, X100)
staining	Enum (HE/ZN)	SEDANG	Pewarnaan: Hematoksilin-Eosin atau Ziehl-Neelsen
qc_status	Enum QcStatus	SEDANG	Status Quality Control: PENDING / PASSED / FAILED

2.1.3 Data Hasil Analisis AI - Model: AiResult & AiFinding

Field	Sensitivitas	Keterangan
total_necrosis_percent	SANGAT TINGGI	Persentase area nekrosis terdeteksi AI
global_severity (SeverityLevel)	SANGAT TINGGI	Tingkat keparahan global: SANGAT_RENDAH hingga SANGAT_TINGGI
segmentation_mask (JsonB)	SANGAT TINGGI	Data segmentasi / bounding box temuan patologis — disimpan sebagai JSONB dengan GIN index
mean_confidence	TINGGI	Rata-rata confidence score model AI

2.1.4 Data Kredensial & Akun - Model: User

Field	Sensitivitas	Keterangan
password_hash	SANGAT TINGGI	Hash bcrypt TIDAK pernah disimpan plaintext. Salt factor = 10.
email	SANGAT TINGGI	Email unik sebagai identifier login pengguna
role (enum Role)	TINGGI	Peran pengguna menentukan hak akses RBAC
sip_number	TINGGI	Nomor Surat Izin Praktik dokter
is_first_login	SEDANG	Flag wajib ganti password pada login pertama
is_active	SEDANG	Status aktif akun / non-aktif menggantikan hard delete

2.1.5 Data Audit & Laporan Klinis

Model	Sensitivitas	Keterangan
AuditLog	SANGAT TINGGI	Rekam jejak semua aksi append-only, tidak dapat dihapus. Field: user_id, action, entity_type, entity_id, payload (JsonB), created_at.

Report	SANGAT TINGGI	Laporan klinis final dengan tanda tangan digital. Field: digital_signature, is_signed, signed_at, pathologist_notes, diagnosis_summary.
Validation	SANGAT TINGGI	Hasil validasi dokter patologi per citra. Berisi global_severity, necrosis_severity, granuloma_severity dalam SeverityLevel.
Consensus	TINGGI	Ringkasan konsensus akhir per kasus diagnosis TBC.

2.2 Aset Kode & Infrastruktur

Aset	Sensitivitas	Keterangan
JWT_SECRET (env var)	SANGAT TINGGI	Kunci penandatanganan token JWT. Jika bocor, seluruh sesi dapat dipalsukan.
SUPABASE_SERVICE_ROLE_KEY	SANGAT TINGGI	API key dengan hak penuh ke Supabase (database + storage). Disimpan di server-side saja.
DATABASE_URL / DIRECT_URL	SANGAT TINGGI	Connection string PostgreSQL via PgBouncer dan koneksi langsung untuk migrasi.
prisma/schema.prisma	TINGGI	Skema struktur seluruh database — blueprint sistem.
src/utils/ (JWT, hash, access)	TINGGI	Modul-modul kriptografi dan kontrol akses inti sistem.
Supabase Storage Bucket	SANGAT TINGGI	Bucket 'histopatologi' menyimpan file citra medis, akses via signed URL dengan TTL terbatas.

3. FUNGSI-FUNGSI KEAMANAN YANG DIGUNAKAN

3.1 Fungsi Kriptografi: Hashing Password (bcrypt)

Library yang digunakan: bcrypt v6.0.0. Fungsi ini terdapat pada file src/utils/hash.utils.ts.

Fungsi	Return Type	Keterangan
bcrypt.genSalt(10)	Promise<string>	Generate salt acak dengan cost factor 10. Semakin tinggi cost factor, semakin lama proses hashing (brute-force resistance).
bcrypt.hash(password, salt)	Promise<string>	Menghasilkan hash bcrypt yang mengandung: versi algoritma, cost factor, salt (22 karakter), dan ciphertext.
bcrypt.compare(input, hash)	Promise<boolean>	Memverifikasi password input dengan hash tersimpan tanpa perlu menyimpan salt secara terpisah (salt embedded dalam hash).

```
// src/utils/hash.utils.ts
export const hashPassword = async (password: string): Promise<string> => {
  const salt = await bcrypt.genSalt(10); // cost factor = 10
  return bcrypt.hash(password, salt);
};

export const comparePassword = async (password: string, hash: string):
Promise<boolean> => {
  return bcrypt.compare(password, hash); // timing-safe comparison
};
```

Format hash bcrypt yang dihasilkan mengikuti pola: `const salt = await bcrypt.genSalt(10);`. Cost factor 10 menghasilkan sekitar $2^{10} = 1024$ iterasi hashing, membuat *brute-force attack* secara komputasional menjadi tidak efisien.

3.2 Fungsi JWT (JSON Web Token)

Library yang digunakan: jsonwebtoken v9.0.3. Fungsi ini terdapat pada file src/utils/jwt.utils.ts.

Fungsi	Expiry Default	Keterangan
generateToken(payload, expiresIn)	"1d" (1 hari)	Menandatangani JWT dengan HS256 menggunakan JWT_SECRET dari environment variable. Token login berlaku 1 hari, token reset password 15 menit.
verifyToken<T>(token)	—	Memverifikasi signature dan expiry JWT. Melempar AppError(401) jika token tidak valid atau sudah kedaluwarsa.
getJwtSecret()	—	Mengambil JWT_SECRET dari process.env. Melempar AppError(500) jika env var tidak tersedia, mencegah sistem berjalan tanpa konfigurasi keamanan.

Token JWT Login mengandung payload: id (UUID pengguna), role (enum Role), dan is_first_login (boolean). Token Reset Password mengandung payload: id (UUID pengguna) dan purpose: "reset_password" untuk mencegah token hijacking lintas fungsi.

3.3 Fungsi Validasi Input (Zod Schema)

Library yang digunakan: Zod v4.3.6. Implementasi terdapat pada direktori src/validations/ dan src/middlewares/validate.middleware.ts.

Schema / Method	Rule Validasi yang Diterapkan
loginSchema	email: z.email(), format email valid + trim + toLowerCase. password: z.string().min(1)
newPasswordSchema	z.string().min(8) + .refine(), wajib kombinasi huruf [A-Za-z] DAN angka [0-9]. Menolak password hanya huruf atau hanya angka.
updateCredentialSchema	.refine() cross-field: newPassword === confirmPassword. Menolak request jika konfirmasi tidak cocok.
resetPasswordSchema	token: z.string().min(1) + newPassword menggunakan newPasswordSchema yang sama.

validate() middleware schema.parse(req.body) - data yang tidak valid ditolak di middleware layer SEBELUM masuk controller/service. Response 400 dengan pesan error dari Zod issue pertama.

3.4 Fungsi Presigned URL Supabase Storage

File: src/utils/supabase-storage.utils.ts. Mengelola akses file citra histopatologi medis tanpa mengekspos file secara publik.

Fungsi	TTL (Time-to-Live)	Kegunaan
createPresignedUploadUrl(filePath)	3600 detik (1 jam)	URL sekali pakai untuk upload citra langsung ke Supabase Storage dari client. Mencegah upload tanpa otorisasi.
createSignedViewUrl(filePath)	900 detik (15 menit)	URL sementara untuk melihat citra. Harus di-request ulang setelah 15 menit, membatasi window eksposur data medis.
deleteFile(filePath)	—	Hapus file dari storage. Dipanggil saat gambar dihapus atau pembersihan stale PENDING images (TTL 2 jam).
buildFilePath(caseId, imageId, name)	—	Membangun path terstruktur: cases/{caseId}/{imageId}/{filename} untuk isolasi dan auditabilitas per kasus.

3.5 Fungsi Kontrol Akses Berbasis Institusi

File: src/utils/access.utils.ts. Mengimplementasikan isolasi data antar institusi kesehatan.

```
// src/utils/access.utils.ts
export const assertSameInstitution = async (userAId: string, userBId: string) =>
{
  const [userA, userB] = await Promise.all([
    prisma.user.findUnique({ where: { id: userAId }, select: { institution: true } }),
    prisma.user.findUnique({ where: { id: userBId }, select: { institution: true } }),
  ]);

  if (!userA?.institution || !userB?.institution)
    throw new AppError('Akses ditolak: institusi tidak terdaftar', 403);

  if (userA.institution !== userB.institution)
```

```
    throw new AppError('Akses ditolak: resource milik institusi lain', 403);  
};
```

Fungsi ini dipanggil dari `image.service.ts` dan `services` lain setiap kali `OPERATOR_LAB` mengakses resource kasus. Memastikan data satu institusi tidak bisa diakses operator dari institusi berbeda.

4. IMPLEMENTASI KEAMANAN PASSWORD

4.1 Strategi Penyimpanan Password

Sistem tidak pernah menyimpan password dalam bentuk plaintext. Seluruh password di-hash menggunakan bcrypt dengan Adaptive Hashing, yaitu algoritma hashing yang dirancang khusus untuk password.

4.2 Alur Lengkap Manajemen Password

4.2.1 Pembuatan Akun (Seed/Admin)

- Akun tidak dapat dibuat melalui endpoint publik (tidak ada POST /register).
- Admin membuat akun langsung ke database menggunakan seed script (prisma/seed.ts).
- Password di-hash menggunakan `hashPassword()` sebelum disimpan.
- Flag `is_first_login` diset true — memaksa pengguna ganti password pada login pertama.

4.2.2 First Login: Wajib Ganti Password

```
// src/services/auth.service.ts — updateCredential()
// 1. Verifikasi password lama
const isPasswordValid = await comparePassword(currentPassword,
user.password_hash);
if (!isPasswordValid) throw new AppError('Email atau password tidak valid',
401);

// 2. Password baru tidak boleh sama dengan password lama
if (await comparePassword(newPassword, user.password_hash))
  throw new AppError('Password baru tidak boleh sama dengan password saat
ini', 400);

// 3. Simpan hash baru, reset flag first login
await prisma.user.update({
  data: { password_hash: await hashPassword(newPassword), is_first_login:
false }
});
```

4.2.3 Reset Password (Stateless JWT-Based)

Reset password tidak memerlukan penyimpanan token di database, melainkan menggunakan JWT dengan claims khusus:

```
// src/services/auth.service.ts – forgotPassword() & resetPassword()

// Step 1: Generate reset token (TTL 15 menit)
const payload: PasswordResetTokenPayload = {
  id: user.id,
  purpose: 'reset_password' // claim khusus – mencegah token hijacking
};
const resetToken = generateToken(payload, '15m');

// Step 2: Verifikasi saat reset
const decoded = verifyToken<PasswordResetTokenPayload>(token);
if (decoded.purpose !== RESET_PASSWORD_PURPOSE) // validasi purpose claim
  throw new AppError('Token reset password tidak valid', 401);
```

4.3 Aturan Password (Validation Rules)

Aturan	Implementasi	Tujuan Keamanan
Minimum 8 karakter	z.string().min(8, ...)	Menolak password terlalu pendek yang mudah ditebak
Wajib huruf + angka	/[A-Za-z]/.test() && /[0-9]/.test()	Meningkatkan entropi password, menolak password all-numeric atau all-alpha
Tidak boleh sama dengan password lama	bcrypt.compare(new, oldHash)	Mencegah password reuse, menghindari serangan replay berbasis credential lama
Konfirmasi password harus cocok	.refine() cross-field validation	Mencegah typo pada password baru yang tidak disadari pengguna
Reset token berlaku 15 menit	generateToken(payload, '15m')	Window eksposur token reset yang minimal, kedaluwarsa otomatis

5. PROTOKOL AAA (Authentication, Authorization, Accounting)

Protokol AAA merupakan kerangka keamanan tiga lapis untuk mengelola akses sumber daya sistem. Sistem TBC ini mengimplementasikan ketiga komponen AAA secara menyeluruh.

5.1 Authentication (Autentikasi)

Autentikasi adalah proses verifikasi identitas pengguna. Sistem menggunakan mekanisme Stateless JWT Authentication.

5.1.1 Mekanisme Login

```
// src/services/auth.service.ts – loginUser()
const user = await prisma.user.findUnique({ where: { email } });

// Cek 3 kondisi sekaligus (timing-safe: tidak reveal info mana yang salah)
if (!user || !user.is_active || !(await comparePassword(passwordInput,
user.password_hash))) {
  throw new AppError('Email atau password tidak valid', 401);
}

// Generate JWT dengan payload minimal
const token = generateToken({ id: user.id, role: user.role, is_first_login:
user.is_first_login }, '1d');
```

Penggabungan pengecekan tiga kondisi dalam satu blok bersifat intentional sebagai perlindungan terhadap username enumeration attack, penyerang tidak dapat mengetahui apakah email terdaftar, akun non-aktif, atau password salah.

5.1.2 Middleware Autentikasi

```
// src/middlewares/authenticate.middleware.ts
export const authenticate = (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (!authHeader?.startsWith('Bearer '))
    return res.status(401).json({ message: 'Token autentikasi tidak ditemukan' });

  const token = authHeader.split(' ')[1];
```

```

    req.user = verifyToken<LoginTokenPayload>(token); // throws 401 if
invalid/expired
    next();
};

```

Komponen	Detail Implementasi
Algoritma Token	HS256 (HMAC-SHA256) - JWT ditandatangani simetris dengan JWT_SECRET
Token Lifetime	Login token: 1 hari (1d). Reset password token: 15 menit (15m).
Token Location	HTTP Authorization header: Bearer {token}. Tidak di-store di server (stateless).
Payload Claims	Login: { id, role, is_first_login }. Reset: { id, purpose: 'reset_password' }.
Secret Storage	JWT_SECRET disimpan di environment variable — tidak hardcoded dalam source code.

5.2 Authorization (Otorisasi)

Otorisasi adalah proses penentuan hak akses setelah identitas terverifikasi. Sistem mengimplementasikan dua lapisan otorisasi: RBAC dan Institution-based Access Control.

5.2.1 Role-Based Access Control (RBAC)

```

// src/middlewares/authenticate.middleware.ts
export const authorize = (...roles: Role[]) => (req, res, next) => {
  if (!req.user || !roles.includes(req.user.role as Role)) {
    return res.status(403).json({
      message: 'Anda tidak memiliki akses ke resource ini'
    });
  }
  next();
};

```

```

// Contoh penggunaan pada route:
router.get('/queue', authenticate, authorize(Role.DOKTER_PATOLOGI),
reviewController.getQueue);
router.post('/', authenticate, authorize(Role.OPERATOR_LAB),
patientController.createPatient);

```

5.2.2 Matrix Otorisasi per Endpoint

Endpoint	OPERATOR_ LAB	DOKTER_PA TOLOGI	ADMIN_AI
POST /api/auth/login	✓ (publik)	✓ (publik)	✓ (publik)
POST /api/patients	✓	✗ 403	✗ 403
POST /api/cases/:id/images/presigned-urls	✓	✗ 403	✗ 403
GET /api/review/queue	✗ 403	✓	✗ 403
POST /api/images/:id/validate	✗ 403	✓	✗ 403
POST /api/cases/:id/consensus	✗ 403	✓	✗ 403

5.2.3 Institution-Based Access Control

Lapisan otorisasi kedua yang lebih granular: OPERATOR_LAB hanya dapat mengakses data pasien dan kasus dari institusi yang sama. Implementasi via `assertSameInstitution()` di service layer.

Kondisi	Perilaku Sistem
Institusi sama	✓ Akses diizinkan — operasi dilanjutkan
Institusi berbeda	✗ 403 Forbidden — 'Akses ditolak: resource milik institusi lain'
Institusi null / tidak terdaftar	✗ 403 Forbidden — 'Akses ditolak: institusi tidak terdaftar'

5.3 Accounting (Akuntabilitas)

Accounting adalah proses pencatatan seluruh aktivitas sistem untuk keperluan audit, forensik digital, dan non-repudiation. Sistem mengimplementasikan tiga mekanisme accounting:

5.3.1 Audit Log (Append-Only)

Model AuditLog di database dirancang sebagai immutable append-only log:

```
// prisma/schema.prisma
model AuditLog {
  id      String  @id @default(uuid())
```

```

user_id   String           // siapa yang melakukan aksi
action    String           // aksi apa (CREATE_CASE, VALIDATE_IMAGE, dll)
entity_type String         // entitas apa (Case, Image, Validation, dll)
entity_id String           // ID entitas yang terdampak
payload   Json? @db.JsonB   // konteks lengkap aksi (diff data)
created_at DateTime @default(now()) // kapan terjadi

// Index untuk query audit yang efisien
@@index([user_id, created_at])
@@index([entity_type, entity_id])
@@index([payload(ops: JsonbPathOps)], type: Gin) // GIN index untuk query
JSONB
}

```

Properti Audit Log	Nilai / Penjelasan
Operasi yang diizinkan	INSERT saja, tidak ada UPDATE atau DELETE pada entri audit log
Non-repudiation	Setiap aksi terikat pada user_id yang spesifik — tidak dapat disangkal
Granularitas	Per-entitas (per Case, per Image, per Validation) dengan payload JSONB
Kemampuan forensik	Index GIN pada payload JSONB memungkinkan query forensik: 'siapa mengubah apa pada kasus X'

5.3.2 Tanda Tangan Digital pada Laporan

Model Report menyediakan infrastruktur non-repudiation untuk dokumen klinis:

```

// prisma/schema.prisma — model Report
model Report {
  // ...
  is_signed      Boolean @default(false)
  digital_signature String? // nilai tanda tangan digital (hash / asymmetric
signature)
  signed_at      DateTime? // timestamp penandatanganan
  generated_by   String // dokter patologi yang menandatangani
}

```


Implementasi lengkap tanda tangan digital tersedia pada modul 03_Source_Code/digital_signature/. Modul ini menggunakan kriptografi asimetris (public-private key) untuk memastikan integritas dan keaslian laporan klinis setiap laporan yang telah ditandatangani tidak dapat dimodifikasi tanpa invalidasi tanda tangan.

5.3.3 Soft Delete & Content Masking

Sebagai bagian dari data integrity untuk rekam medis, sistem mengimplementasikan soft delete pada komentar klinis:

```
// src/services/comment.service.ts
// Komentar tidak dihapus secara fisik — jejak tetap ada di database
export const maskDeletedContent = (comment) =>
  comment.is_deleted ? { ...comment, content: 'pesan ini telah dihapus' } :
  comment;

// Konsekuensi: rekam medis tetap utuh untuk keperluan audit dan hukum
// Field is_deleted = true, content termasuk histori aksi tetap tersimpan
```

5.4 Ringkasan Pemetaan Protokol AAA ke CIA Triad

AAA	CIA Triad	Implementasi dalam Sistem
Authentication	Confidentiality	JWT HS256 + bcrypt adaptive hashing. Stateless reset token dengan purpose claim. Cegah username enumeration.
Authorization	Confidentiality + Integrity	RBAC (3 roles) via middleware chain authenticate → authorize. Institution-based data segregation. Presigned URL dengan TTL untuk citra medis.
Accounting	Integrity + Availability	Append-only AuditLog dengan GIN index. Soft delete rekam medis. Tanda tangan digital laporan klinis. Content masking komentar.

5.5 Prinsip Zero Trust dalam Desain Sistem

Sistem menerapkan prinsip Zero Trust dengan menghilangkan attack surface registrasi publik:

- Tidak ada endpoint POST /register — akun hanya dibuat oleh admin langsung ke database.

- Setiap request yang membutuhkan akses resource diverifikasi ulang identitas dan otorisasinya.
- Akun non-aktif (is_active: false) ditolak pada login, menggantikan hard delete untuk menjaga integritas jejak audit.
- CORS dikonfigurasi dengan whitelist origin spesifik dari environment variable, menolak semua origin yang tidak terdaftar.